



Examiner Note

The ZIP file contains:

- `solution.pdf` : Full pdf of the solutions for the entire exam
- `Q1` : Directory for Q1
- `Q2` : Directory for Q2
- `Q3` : Directory for Q3
- `Q4` : Directory for Q4

In all `Q` directories are the full work done. In `Q1` and `Q2` there are only written files as they required no coding. In `Q3` and `Q4`, there are the full source code, header files and scripts used. Each directory also contains a pdf relating to the question, however all the same information is in `solution.pdf`. For the coding questions, the individual solutions will guide you through the rest of the directory.

For Q4, it is important to note that the source code is in `src` for both serial and parallel (each having the majority of the work done in `main.cpp`) and the header files are in `include` both for serial and parallel. I highly recommend using the `Makefile` if you will be compiling and executing binaries as it will take all this into account.

All solutions were written in Markdown for ease of transferring things like code snippets and images. However, the formatting on turning a Markdown file to a PDF file means that there are a lot of pages to this. Everything that is written is relevant, and the number of pages is due to the amount of space that the formatting left to fit everything in coherently.

Q1

	ARCHER2	Bede
Nodes	5,860	32
Processor	2× AMD EPYCTM 7742, 2.25 GHz, 64-core	2x, IBM Power9, 2.7 GHz, 16-core
Cores per node	128 (2 x 64-core processor)	32 (2x 16-core processor)
Memory per node	256 GiB (standard memory), 512 GiB (high memory)	512GB DDR4 RAM
Interconnect	HPE Cray Slingshot, 2× 100 Gbps bi-directional per node	NVLink 2.0
GPU per node	Only 4 nodes with 4x AMD Instinct MI210	4x NVIDIA Tesla V100 32GB NVLink 2.0 GPUs (2 per CPU)

It is clear from the table that ARCHER2 is focused on a CPU architecture, with 750,080 cores in total. ARCHER2 has an estimated peak performance of 28 PFLOPs. However, Bede is focused on a GPU architecture, boasting many more high powered GPUs than ARCHER2.

- ARCHER2 is designed for general purpose high performance computing, running programs such as CASTEP and HadGEM3. The use of 64 core processors and a large amount of nodes means that ARCHER2 offers extreme benefits from parallel programming. On the other hand, Bede is designed for more specialised applications that need GPU resources such as machine learning, a more modern demand.
- For large parallel programs, the communication between different pieces of hardware is extremely important. ARCHER2 has a 100 Gbps interconnect which is quite fast and does help minimise communication time. However Bede's NVLink 2.0 offers a 25 Gigatransfers per second, which is an estimated 1600 Gbps. This much higher interconnect is needed for the high memory usage that GPU programs typically need.

- ARCHER2 has a focus on scalability that programs from a variety of disciplines can be ran on it, making it a central computational resource for the wider country. However, Bede has more of a focus on specialised studies such as AI, machine learning and neural networks and the decision to focus on GPUs gives researchers a great computational resource for their own studies.

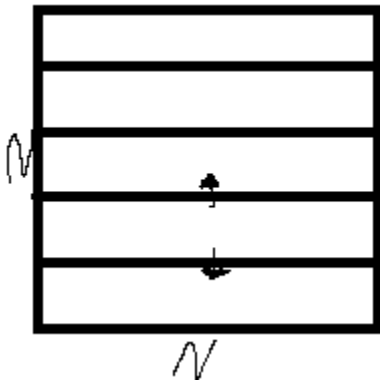
For GROMACs, both supercomputers offer their own benefits to being used. However I think that Bede would be more suitable for a GROMACs simulation as with all molecular dynamic simulations, there is a heavy cost of computation on solving Newton's equations. For a large number of particles, there will be a lot of computation necessary, to which GPU acceleration excels at. Another benefit to using Bede is the high memory interconnect, which means the communication between the heavy loaded GPU and CPU is still small compared to the size of the simulation. ARCHER2 does have a large amount of compute nodes, however due to the memory interconnect and the overhead associated with running the nodes, I expect that ARCHER2 would perform a lot worse for large simulations compared to Bede.

Q2

Assumptions

- 2D data set with N_{grid} total grid points, N on each side.
- χ FLOPs per iteration for each grid point.
- P processors, that can perform S FLOPs a second.
- A latency L and bandwidth B .
- Σ grid points on boundaries between partitions.
- Only one comm operation per iteration
- No overlap between comms and computation
- Size of each message is σ bytes per grid point.
- Periodic boundary conditions

1D Data Decomposition



For 1D decomposition, the grid is split into strips of size $\frac{N^2}{P}$. There are only 2 communications per iteration per strip, and each boundary has N points.

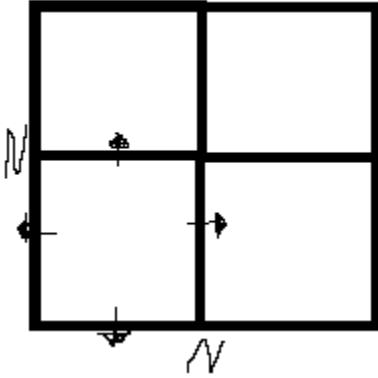
Hence, $\Sigma = 2N$.

Figure of Merit, $F = \frac{P\Sigma}{N_{grid}} = \frac{P\Sigma}{N^2} = \frac{2P}{N}$ i.e. the fraction of grid points involved in comms is of order P .

The max number of processors before speed does not increase is given by $P_{max} = \frac{\chi N_{grid} B}{(\sigma \Sigma + N_m B L) S}$ for N_m the number of messages.

For 1D, $P_{max} = \frac{\chi N^2 B}{(2\sigma N + N_m BL)S}$

2D Data Decomposition



For 2D decomposition, the grid is split into subgrids of size $\frac{N^2}{P}$. There are now 4 communications per iteration per subgrid, and each boundary has $\frac{N}{\sqrt{P}}$ points.

Hence, $\Sigma = 4 \frac{N}{\sqrt{P}}$.

Figure of Merit, $F = \frac{P\Sigma}{N_{grid}} = \frac{P\Sigma}{N^2} = \frac{4\sqrt{P}}{N}$ i.e. the fraction of grid points involved in comms is of order $\sqrt{P}$.

$$P_{max} = \frac{\chi N_{grid} B}{(\sigma \Sigma + N_m BL)S}$$

For 2D, $P_{max} = \frac{\chi N^2 B}{(4\sigma N \sqrt{P} + N_m BL)S}$

Efficiency

2D decomposition will become more efficient when its figure of merit is lower than the 1D decomposition.

Hence:

$$\frac{4\sqrt{P}}{N} < \frac{2P}{N}$$

$$P > 4$$

Hence when there are more than 4 processors, the 2D decomposition is more efficient.



Q3

Part a

The profile output for the first run is:

```
%   cumulative   self           self   total
time  seconds  seconds    calls  ms/call  ms/call  name
50.94   375.60   375.60 19121070    0.02    0.04  pbc_loop()
45.18   708.74   333.13 14183404272    0.00    0.00
std::vector<double, std::allocator<double> >::operator[](unsigned long)
 3.71   736.11   27.38 19135071    0.00    0.00
std::vector<double, std::allocator<double> >::size() const
 0.15   737.25    1.14    4000    0.28   183.18  calculate_forces()
 0.01   737.36    0.11    4000    0.03    0.87  move_particles()
```

Note function arguments have been omitted

In total it took around 739.39 seconds for the full run. The biggest bottleneck as identified by the profiler is the `pbc_loop()` below:

```

void pbc_loop(OMMITED_ARGUMENTS){

    const int num_particles = x.size();

    // define function variables
    double rjx, rjy, rjz, rjr; // particle j positions/radii
    double dx, dy, dz, rr; // position vector and radii

    for (int j = 0; j < num_particles; j++){
        rjx = x[j] + offset_x;
        rjy = y[j] + offset_y;
        rjz = z[j] + offset_z;
        rjr = particle_radius[j];
        dx = rjx-rix;
        dy = rjy-riy;
        dz = rjz-riz;
        rr = (rir+rjr)*(rir+rjr);
        // check if particles overlap
        if(dx*dx+dy*dy+dz*dz < rr){
            fx += -force_size*dx;
            fy += -force_size*dy;
            fz += -force_size*dz;
        }
    }
}

```

The program spends a lot of time in this function as it is called many times in other functions (mostly `calculate_forces()`), and is not very optimised. This function performs an overlap check between particle i (which calls the function) and every other particle in the system (particle j). This is then a $O(N^2)$ loop, which is very slow for large number of particles, such as in this case when there are 4000 particles. This could be optimised with a nearest neighbour approach, where only the nearest neighbour particles of i are checked for overlap. However this is not a good candidate for making parallel as it is called many times in `calculate_forces()`, and continually setting up and taking down the threads will incur quite a large overhead. Thus naturally, it is better to make `calculate_forces()` parallel. This will cause great speedup.

```

void calculate_forces()
{
    /* Code */

    for (int i = 0; i < np; i++)
    {
        riz = z[i];
        rix = x[i];
        riy = y[i];
        rir = particle_radius[i];
        fx = 0.0;
        fy = 0.0;
        fz = 0.0;

        /* MAIN CODE */

        force_x[i] = fx;
        force_y[i] = fy;
        force_z[i] = fz;

    } // i-loop
}

```

This function has the most work, thus is the natural candidate for making parallel. OpenMP utilises `omp for` which is used on `for` loops. This work also might be a good candidate for `omp task`, which handles the load bearing of the work across threads better than `omp for`.

The next bottleneck is reading from vectors, as given by the second line in the profile. Work could be done to reduce the number of look ups of vectors, however this is more of a serial optimisation.

Part b

I did two trials of different OpenMP directives for speedup, one for `omp for` and one for `omp task`. The results for this are in the `second_run` and `third_run` directory respectively.

First Trial

```
void calculate_forces(OMITTED_ARGUMENTS)
{

    const int np = x.size();
    const double force_size = 0.1;

    // define function variables
    double rix, riy, riz, rir;           // particle j positions/radii
    double fx, fy, fz;                   // force components
    bool bpx, bmx, bpy, bmy, bpz, bmz; // boolean tests

    // loop over all particles and calculate total force
    #pragma omp parallel for default(shared) private(rix, riy, riz, rir, fx, fy, fz,
    bpx, bmx, bpy, bmy, bpz, bmz)
        for (int i = 0; i < np; i++)
        {
            /* MAIN CODE */

            force_x[i] = fx;
            force_y[i] = fy;
            force_z[i] = fz;
        }
    }
```

This was the major change of the code. There were no race conditions to update the vectors (`force_*`) as each `i` is private to the thread, and each thread will have a different `i` concurrently, thus no two threads are accessing the same part of memory at once.

```

void move_particles(OMITTED_ARGUMENTS)
{

    const double mass = 1.0;
    const int np = x.size();

    // relaxation algorithm - simply move particles along direction of force
    #pragma omp parallel for default(shared)
        for (int i = 0; i < np; i++)
        {
            x[i] = x[i] + fx[i] * dt;
            y[i] = y[i] + fy[i] * dt;
            z[i] = z[i] + fz[i] * dt;
        }

}

void shrink(OMITTED_ARGUMENTS)
{

    const double shrink_factor = (L - dL) / L;
    const int np = x.size();

    // relaxation algorithm - simply move particles along direction of force
    #pragma omp parallel for default(shared)
        for (int i = 0; i < np; i++)
        {
            x[i] = x[i] * shrink_factor;
            y[i] = y[i] * shrink_factor;
            z[i] = z[i] * shrink_factor;
        }

}

```

Similarly here in the `move_particles()` and `shrink()` functions, I used an `omp for` directive. Once again, all the shared data is accessed with a private variable `i` thus there are no race conditions to be concerned about.

Using `Linux` command `diff` on the output for this run against the `first_run`

results, there were no differences. This follows with the fact that there were no race conditions.

The time for this trial was 12.5 seconds, as seen in `second_run/times.txt`. This was ran on `GViking` with 96 threads.

Second trial

```
void calculate_forces(OMITTED ARGUMENTS)
{
    #pragma omp single
    {
        for (int i = 0; i < np; i++)
        {
            #pragma omp task firstprivate(i) shared(force_x, force_y, force_z)
            {
                double riz = z[i];
                double rix = x[i];
                double riy = y[i];
                double rir = particle_radius[i];
                double fx = 0.0;
                double fy = 0.0;
                double fz = 0.0;

                /* MORE CODE */

                force_x[i] = fx;
                force_y[i] = fy;
                force_z[i] = fz;
            }
        }
    }
}
```

This time I ran the code with creating all work done in the loops as `omp tasks`. I did this in an identical way for `calculate_forces()`, `move_particles()` and `shrink()`.

`omp tasks` are better designed for load bearing, where a `single` thread creates

the `task` where the major work is done. The specified `firstprivate(i)` ensures that each task has its own `i` value, so no race conditions occur. Now threads pick up `tasks` from the master thread, which ensures that no threads are idle while completing the loop. However, this incurs a slightly larger overhead than `omp for` so is useful for when the work each thread has to do is differently loaded to each other. For our case however, every task does the same amount of work due to the discussed `O(N^2)` loop. So it is likely that this will be slightly slower than `omp for`.

Using `Linux` command `diff` on the output for this run against the `second_run` results, there were no differences. This follows with the fact that there were no race conditions.

The time for this trial was `13.04` seconds, as seen in `third_run/times.txt`. This was ran on `GViking` with 96 threads.

And as expected, the `omp task` approach is slightly slower. It is likely the case that this would be a lot slower at lower threads due to the overhead of tasks compared to `omp for`.

Thus I used the first trial code, which is what is seen in `packing/packing_parallel.cpp`

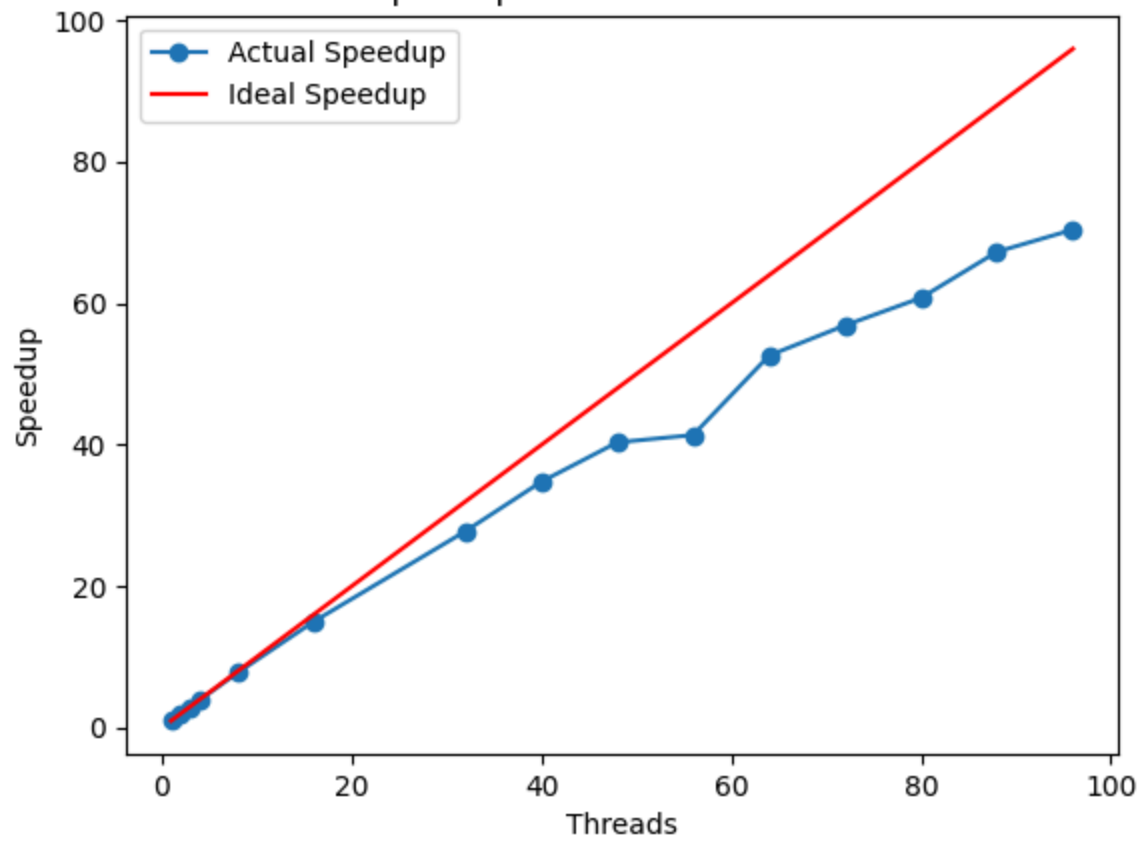
Part c

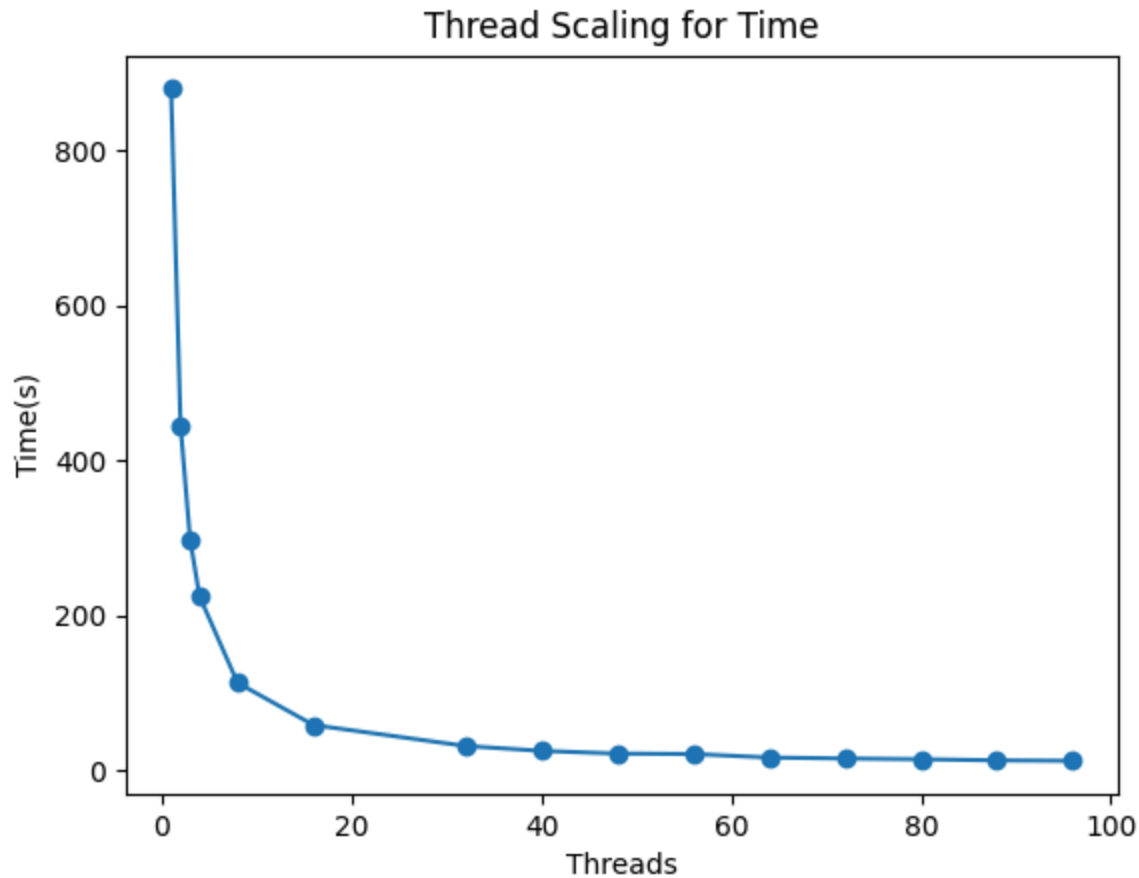
Running on `GViking` with scripts in `run_scripts` and `job_scripts`, I ran a scaling test on the modified code from 1 -> 96 threads. The table below shows the actual time taken and the speedup for each thread run compared to 1 thread. These results can be found in the `data` directory, and also the logs in `logs`. After each run, I compared the output files to `first_run` using `diff` and no differences were seen.

Number of Threads	Time(s)	Speedup
1	879.028	1
2	444.113	1.97928905
3	298.072	2.94904587

Number of Threads	Time(s)	Speedup
4	224.447	3.91641679
8	113.102	7.77199342
16	58.5899	15.0030637
32	31.6238	27.7964065
40	25.2662	34.790669
48	21.7798	40.3597829
56	21.2322	41.4007027
64	16.6987	52.6405049
72	15.4474	56.9045924
80	14.4606	60.7877958
88	13.0613	67.3001922
96	12.4804	70.4326784

Speedup from 1 -> 96 threads





The first graph plots the speedup the code achieved against the possible speedup. This is the theoretical limit of parallel programming, where increasing the number of threads causes the code to speed up that many times, i.e. a linear relation. At quite low threads, it is shown that we are achieving this theoretical speedup. However as the threads increase, there is more of a divergence from the line. This is due to the effects of Amdahl's Law $T(N) = S + \frac{P}{N}$ for S serial execution and P parallel execution. This law shows that at higher and higher threads, the serial execution time will dominate, causing a divergence from the ideal.

Another reason is the communication between threads and memory when accessing or writing to shared data. As more threads are used, there needs to be more communication between memory and the thread, and due to finite bandwidth, the latency will likely dominate the communication time, causing a larger slow down at higher thread numbers.

The second graph shows an exponential decrease in the time taken as the thread numbers increase. This is expected with parallel programming.



Q4

Part a

General Overview

The logic for the serial code (found in `src/src_serial`) is as follows

- Load in the config file (`config/config.txt`) from the user
- Initialise all variables necessary
- Start a `while (true)` loop
- Within the loop, circle centers are generated via a seeded random number generator. Those circles are checked against an `unordered_map` of the other generated circles for overlap.
- If no overlap is found, it is placed within the `unordered_map`
- After the user parameter `sampling_frequency` number of trials, there is a check for convergence. If the number of circles generated has not changed after `sampling_frequency` trials, then there can be no more circles placed without overlap with these parameters and the loop is exited.
- The final results are outputted to screen and files (`coords.bin` and `p_fractions.bin`) and graphs can be plotted (`graphing/graph.py`).

The use of `make` with the rules `serial` , `serial_debug` , `serial_vis` are used to compile the source code. Each rule gives a different binary output with:

- `serial` : Compiles the code without any visualisation or debugging, and outputs final answer.
- `serial_debug` : Compiles the code with custom debugging flags.
- `serial_vis` : Compiles the code with custom visualisation flags, that continually outputs the key data for visualisation purposes (you can watch the numbers go up).

All binaries are put in `bin` when compiled. I highly recommend using the bash script `run_serial.sh` with command `bash ./run_serial.sh <MAKE_RULE>` . This builds

and executes the binary, and will create directories for results to be moved into, which then can be plotted using the Python script `graphing/graph.py`.

The program requires a config file at the path `./config/config.txt`. An example one is below:

```
r 1.234
L 500
sampling_frequency 1000000
seed 18293
```

Specfics

Generation of circles

```
/* Generates and returns a Point, within the boundaries */
Point gen_random_pair(rng &random_gen)
{
    return {
        r + (L - 2 * r) * random_gen.grnd(),
        r + (L - 2 * r) * random_gen.grnd(),
    };
}
```

The generating function will generate a random number (that has been previously seeded) with a number that is within the boundaries of the box. This assures that no circle can overlap the boundary, and reduces the need to run four comparison checks every generation.

Checking overlap

The biggest bottleneck to this problem is the checking the overlap of circles. A naive approach would be a simple comparison between every circle, giving an $O(N^2)$ loop. For efficiency, I implemented a hash map that relates `Cells` to `vector<Point>` i.e I split the grid up into cells that are determined by the input parameters `L` and `r` i.e. `grid_size = static_cast<int>(L / (2 * r));`.

```

// Check if a new circle overlaps with circles in nearby grid cells
bool check_overlap(const Point &new_circle)
{
    Cell cell = get_grid_cell(new_circle);

    const vector<Cell> neighbours = {{-1, -1}, {-1, 0}, {-1, 1},
    {0, -1}, {0, 0}, {0, 1}, {1, -1}, {1, 0}, {1, 1}};

    // Check nearby neighbour cells

    for (auto &neighbour : neighbours)
    {
        Cell neighbor_cell = {cell.first + neighbour.first,
        cell.second + neighbour.second};

        // Check if neighbor cell exists in the spatial grid
        if (grid.find(neighbor_cell) != grid.end())
        {
            for (const auto &existing_circle : grid[neighbor_cell])
            {
                // Compute squared distance for comparison
                double dx = existing_circle.first - new_circle.first;
                double dy = existing_circle.second - new_circle.second;
                double distance_squared = dx * dx + dy * dy;

                if (distance_squared < r_comp)
                {
                    return true; // If overlap, return true
                }
            }
        }
    }

    return false; // No overlap found
}

```

The `check_overlap` function will only compare the circles that are within the nearest neighbour cells. This is a lot more efficient than $O(N^2)$, as a hash map

(with a good hash) is on average $O(1)$ lookup time.

Sampling Frequency approach

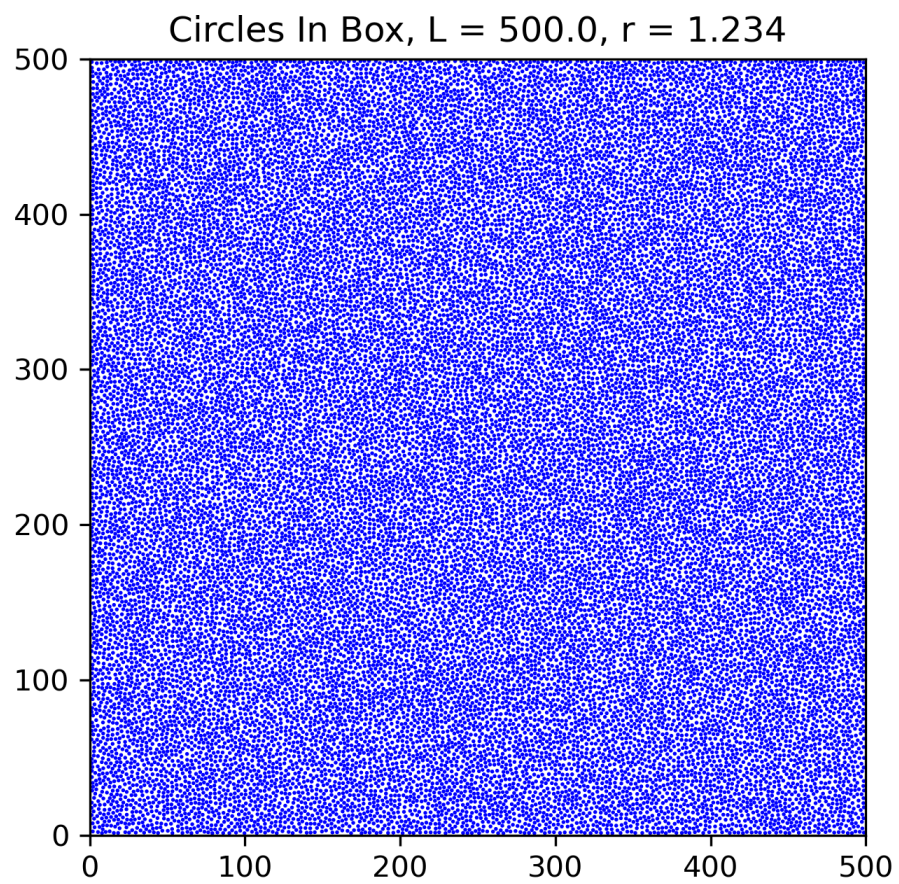
I implemented a `sampling_frequency` approach to the generation of circles, to fit into the idea of "run until no more circles can be placed". Choosing a small `sampling_frequency` will be more efficient, however is very likely to miss a lot of circles that could be generated. On the other hand, choosing a large `sampling_frequency` will be less efficient, but you will likely catch a lot of circles that could be generated. An improvement that could be made is to make the parameter dynamic. I found that a parameter of `1e6` is a good benchmark, however results have been given for `1e6` , `1.25e6` , `1.50e6` , and `1e7` . These can also be found in `results/serial_results/L=500, r=1.234/sf={SAMPLING_FREQUENCY}` . All these results were run on `GViking` with `job_serial.sh` .

Results

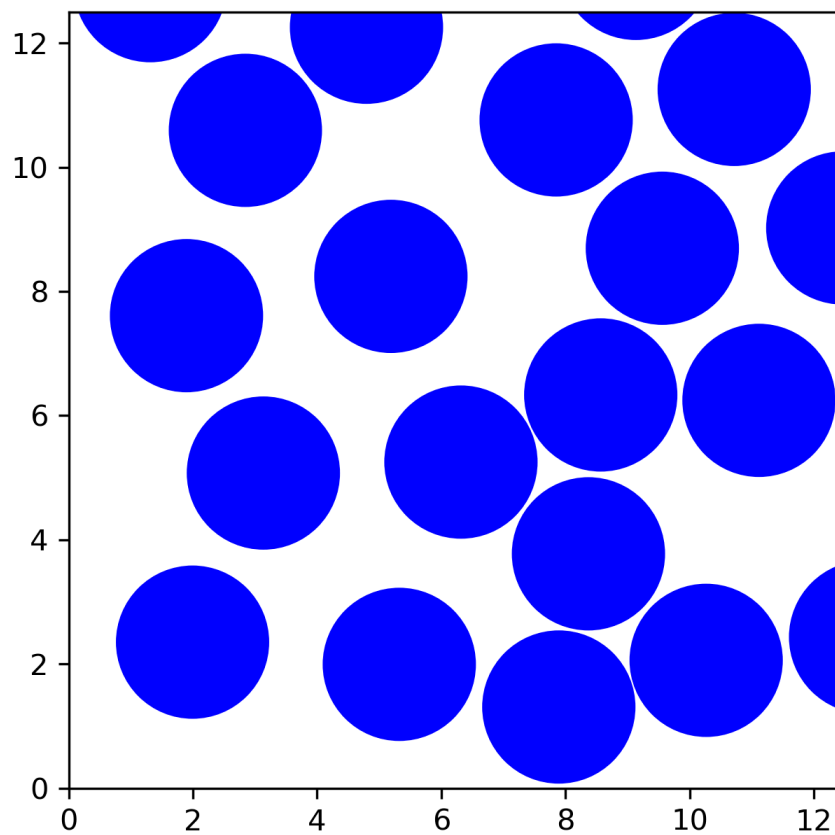
Table

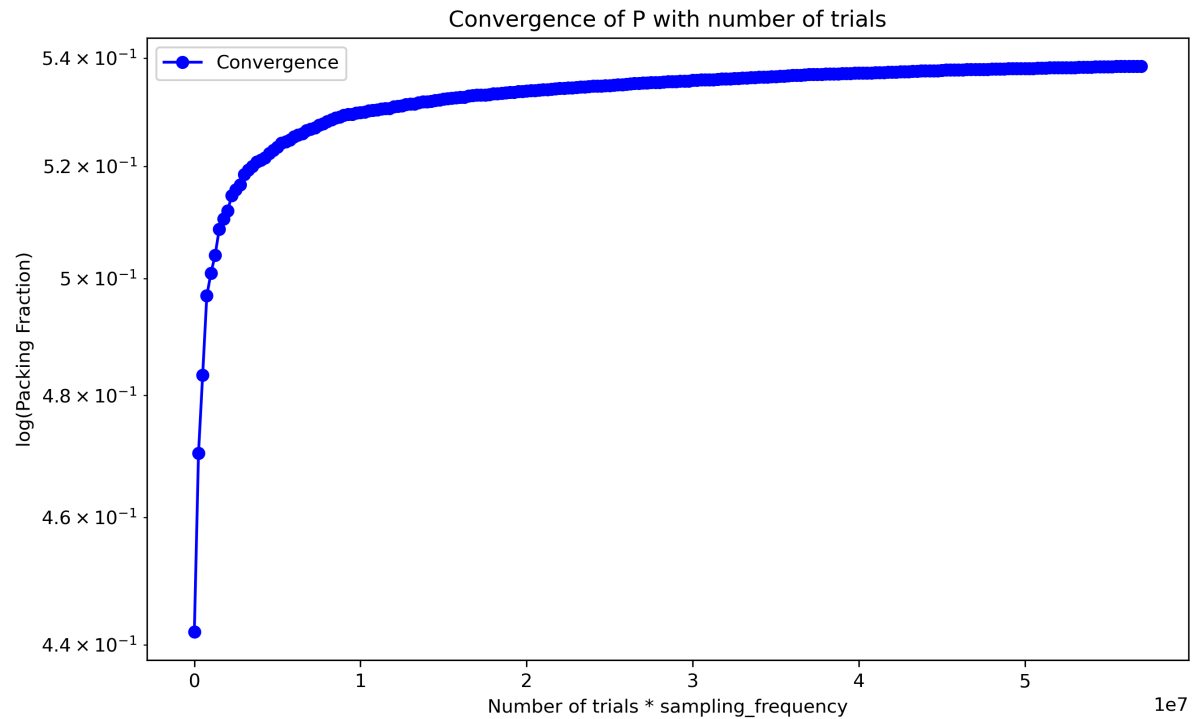
Sampling Frequency	Number of Circles Placed	Packing Fraction	Time(s)
1e6	28141	0.538493	176.691515
1.25e6	28199	0.539602	253.402379
1.50e6	28226	0.540119	290.254791
1e7	28335	0.542205	841.623180

Plots



Circles in Smaller Box, $L = 500.0$, $r = 1.234$





The above plots are for `sampling frequency=1e6`. The plots across all sampling frequencies are very similar, and more can be found in `./results/serial_results`. Note for the convergence plot, at every `sampling_frequency / 4` intervals and after each generation of trials, a point is plotted. This is why the x-axis is labelled `Number of trials * sampling_frequency`.

Part b

General Overview

The logic for the parallel code (found in `src/src_parallel`) is as follows:

- Load in the config file from the user
- Initialise all variables necessary
- Initialise an OpenMP parallel region
- Initialise a thread-local random number generator with a unique seed depending on the thread id and total number of threads.
- Start a `while` loop
- Within the loop, circle centers are generated in parallel, and added to a

new vector `trial_placements` with size `sampling_frequency`. Each circle has a flag associated with it.

- Trials are checked in parallel with the `unordered_map` of placed circles for overlap. If an overlap is found, the trial's flag is set.
- Trials are checked in parallel within themselves, skipping any trials that have flags that are already set.
- Trials that have not got a set flag are placed in the map.
- There is a check for convergence. If the number of circles generated has not changed after `sampling_frequency` trials, then there can be no more circles placed without overlap with these parameters and the master thread calls for all other threads to continue through their loops.
- The necessary data is outputted, as well as written to file.

The use of `make` with the rules `parallel`, `parallel_debug`, `parallel_vis` and `parallel_vik` are used to compile the source code. Each rule gives a different binary output with:

- `parallel`: Compiles the code without any visualisation or debugging, and outputs final answer.
- `parallel_debug`: Compiles the code with custom debugging flags.
- `parallel_vis`: Compiles the code with custom visualisation flags, that continually outputs the key data for visualisation purposes (you can watch the numbers go up).
- `parallel_vik`: Compiles the code with flags custom flag for running on GViking. Should only be compiled on GViking.

All binaries are put in `bin` when compiled. I highly recommend using the bash script `run_parallel.sh` with command

`bash ./run_parallel.sh <MAKE_RULE> <NUM_OF_THREADS>`. This builds and executes the binary, and will create directories for results to be moved into (note: The packing fraction convergence is not calculated here).

The parallel code also accepts a command line argument `--config=PATH_TO_CONFIG`, however will default to the config path `./config/config.txt`. Note `run_parallel.sh` can only accept the default config path. I added the flag for ease of use when doing the ratio scaling tests, and is ran using `run_parallel_viking.sh` and `run_parallel_viking_ratio.sh`. These scripts will not function on a local PC, only on

GViking .

It is important to recognise that the values here for the number of circles are unlikely to match the serial version with the same parameters, due to the fact each thread has a local random number generator, so there are different sequences of numbers being trialed.

Specfics

Why OpenMP

I chose to use OpenMP due to the clear cut logic that is required. Generating random points is a task that can easily be done in parallel, same with read only checking. OpenMP provides easy to use synchornisation in the form of barriers and critical statements, so updating shared resources like the grid is already controlled, and doesn't require special communication like from MPI. OpenMP excels for dynamic workloads, utilising all threads effectively which is needed when checking overlaps.

I did not want to use MPI for splitting up the grid, as for example, if spilt the grid into chunks and had each chunk generate circles, there will likely be an equal distribution across the boundaries of the chunks. I also did not think there would be a benefiet to using a shared memory approach across different processors, as all the memory necessary can be on one node.

Generation of circles

```
rng local_random_gen;
int tid = omp_get_thread_num();
// Unique seed per thread
local_random_gen.seed(tid * seed + omp_get_num_threads());

while (!done)
{
    // Parallel loop to generate trials
    #pragma omp for schedule(static)
    for (int i = 0; i < sampling_frequency; i++)
    {
        if (done)
            continue; // Check shared flag to stop early
        // Generate new trial circle
        Point new_circle = gen_random_pair(local_random_gen);
        // Store it globally
        trial_placements[i] = {new_circle, false};
    }

    /* Sync all threads, ensuring that trials are
    generated before checking overlaps */
    #pragma omp barrier

    // More code
}
```

The generation of circles is now in parallel, where each thread has a differently seeded random number generator. Due to this, changing the number of threads will change the number of circles that are eventually placed, as with each new thread used there are more sequences of numbers that are being trialed. This could have been avoided with only using a single thread to generate numbers, however in that case, all other threads are not being used effectively for smaller grids. Each trial has a flag that starts as `false`, which will be set if any overlap is found.

By specifying the scheduling of the loop to `static` this guarantees consistency across runs with the same number of threads, however does add a slight overhead

as now some threads will be idle.

Checking overlap

With the introduction of generating circles in parallel, there is a need for another check that the generated circles themselves do not overlap. This was not a problem in the serial code as there was only one circle generated at a time.

```

// **Overlap Check Between Trials and Grid**
#pragma omp for schedule(dynamic)
    for (int i = 0; i < sampling_frequency; i++)
    {
        if (done)
            continue; // Check shared flag to stop work early

        check_overlap(trial_placements[i]);
    }

// Sync all threads after overlap check
#pragma omp barrier

// **Overlap Check Between Trials Internally**
// All trials generated by all threads must be compared with each other.
#pragma omp for schedule(static)
    for (int i = 0; i < sampling_frequency; i++)
    {
        if (done)
            continue; // Check shared flag to stop early

        if (trial_placements[i].second)
            continue; // Skip raised flag already

        // Compare each trial with others
        for (int j = i + 1; j < sampling_frequency; j++)
        {
            // Compare trial[i] with trial[j] for overlap
            Point circle1 = trial_placements[i].first;
            Point circle2 = trial_placements[j].first;

            double dx = circle2.first - circle1.first;
            double dy = circle2.second - circle1.second;
            double distance_squared = dx * dx + dy * dy;

            if (distance_squared < r_comp)
            {
                // Set overlap flag for one trial if they
                // overlap, the other is still possible trial
            }
        }
    }

```

```

        trial_placements[i].second = true;
        break;
    }
}

#pragma omp barrier

// More code

```

The `check_overlap()` function is effectively the same as before, but now threads call it in parallel to check. This only reads the grid, so is safe to do in parallel. This also means the scheduling can be `dynamic`, which improves the load bearing. If there is an overlap found, the flag on the circle is set. As each thread will have it's own unique trial placement to set the flag on, this is also thread safe.

After a synchronisation, there is an internal check between trials, in the form of $O(N^2)$ loop. To avoid the cost of this however, we skip any set flags. Each thread has it's own unique sequences to check within the trials. The implementation of similar grid logic as the serial version was tried here, however it was very difficult to make it thread safe and caused more slow down, as all the trials needed to be placed in another grid first. The $O(N^2)$ loop does guarantee that no circles can overlap. However the schedule for this also has to be `static` to guarantee consistency. This occurs more of an slow down as now some threads may have larger loads, however is necessary for consistency.

Placing the circles

```
// Place the trials in the grid based on their flag
#pragma omp for schedule(dynamic)
for (int i = 0; i < sampling_frequency; i++)
{
    if (done)
        continue;

    if (!trial_placements[i].second) // No overlap
    {

        // Enter critical section to update shared resources
        #pragma omp critical
        {
            Point new_circle = trial_placements[i].first;
            // Add to circle coordinates
            circle_coords.push_back(new_circle);
            // Place circle in the grid
            place_circle(new_circle);
        }
    }
}

// More code
```

This is the only critical section of the code due to adding to the grid.

Scaling Tests

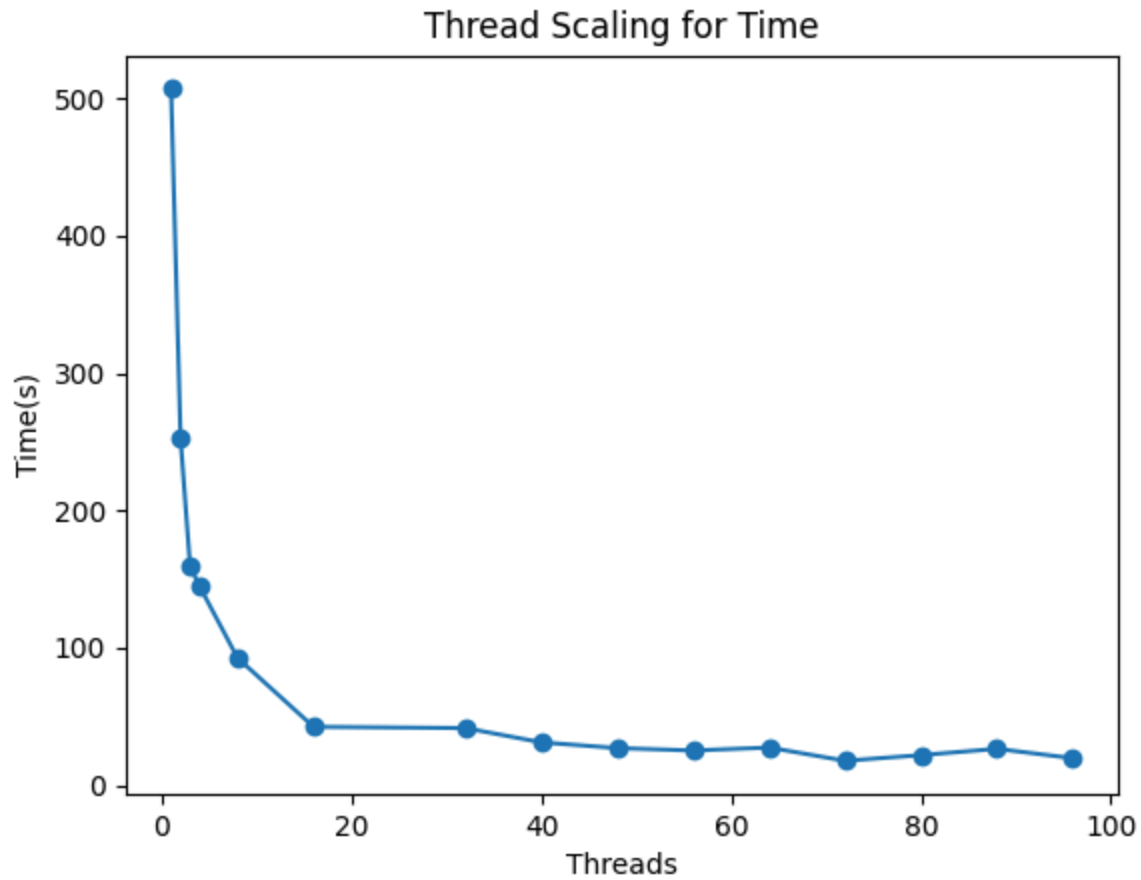
Thread Scaling

I ran the thread scaling test on `GViking` with user parameters:

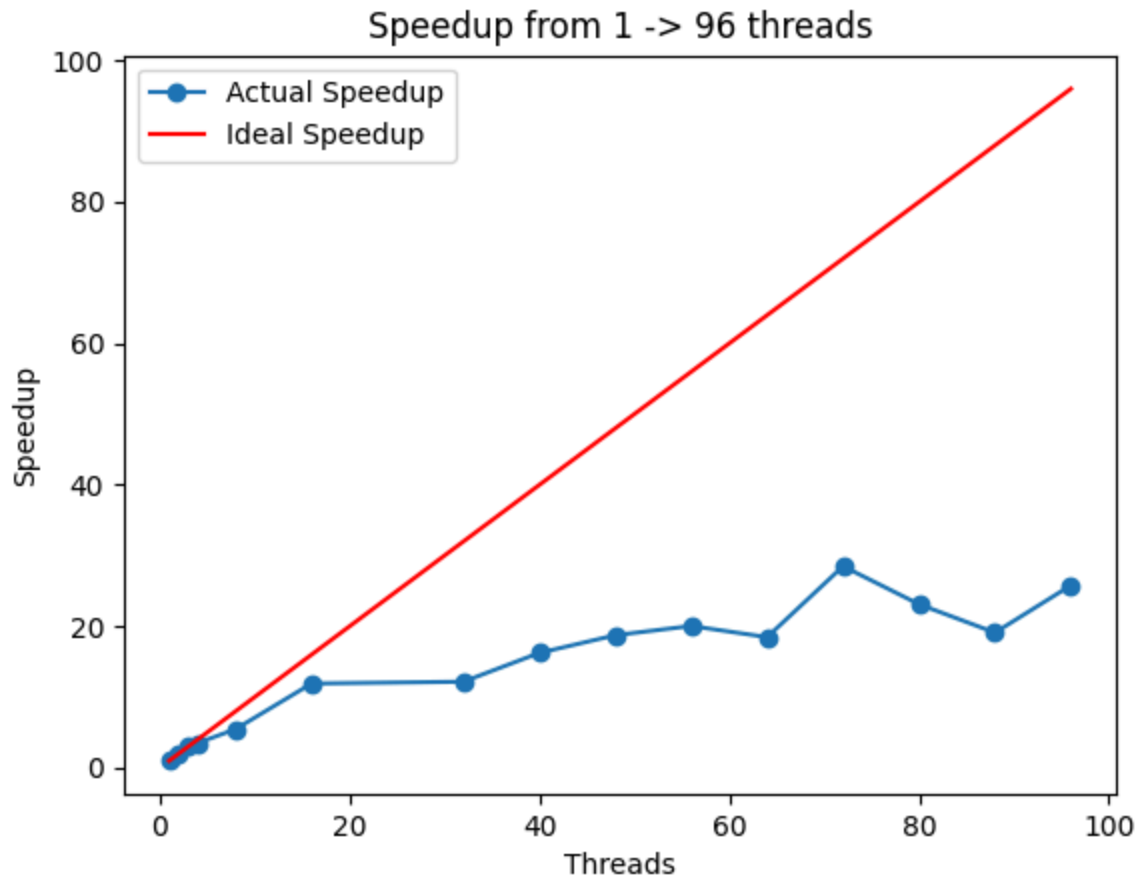
```
L = 500
r = 1.234
sampling_frequency = 1000000 = 1e6
seed = 18293
```

using `job_parallel.sh` and `run_parallel_viking.sh`. These scripts control the changing of the number of threads, creating the directories for results and using `make` to `make parallel` or `make parallel_vik` (depending on use). All relevant flags can be found in the Makefile. I decided to run on multiples of 8 threads, after the initial 4.

Number of Threads	Time(s)	Speedup compared to 1 thread
1	507.1868	1.00
2	252.4379	2.01
3	159.5824	3.18
4	144.9173	3.50
8	92.74313	5.47
16	42.5562	11.92
32	41.62602	12.18
40	31.21906	16.25
48	27.05239	18.75
56	25.28407	20.06
64	27.44813	18.48
72	17.79633	28.50
80	21.92479	23.13
88	26.5281	19.12
96	19.65813	25.80



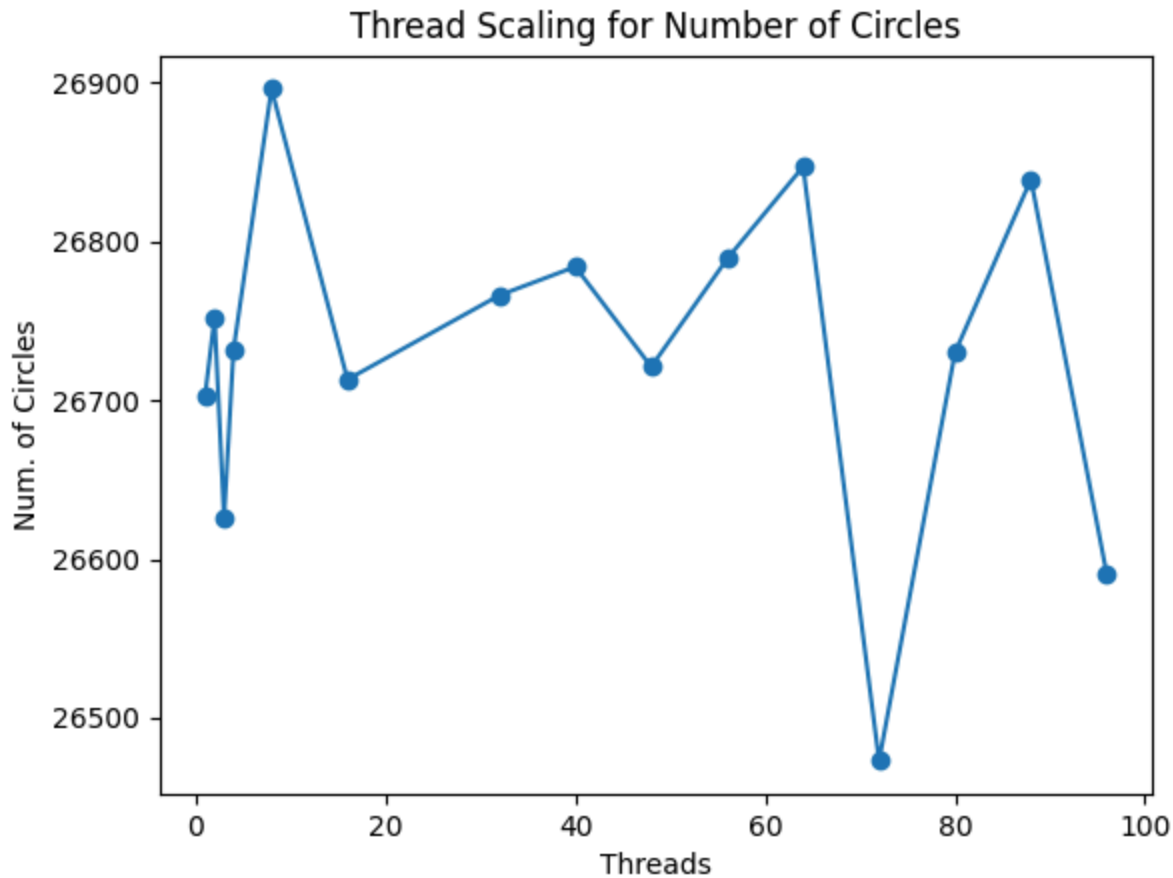
The graph above shows the mean times for the run. The graph shows that the speedup is roughly exponential, which is expected for parallelism. The speed up from 1 thread to 96 threads is $\sim 25.8x$ for these parameters.



However we can quite quickly see the ideal speedup diverges from the actual speedup for these parameters. This might indicate the idea of Gustafson's law where the problem may not be big enough to get the most parallel efficiency. This is more dependent for our case on the `sampling_frequency` , but also for the parameters of `L` and `r` . We will see in the ratio tests that there are more maximal speedups that we can achieve.

Number of Threads	Number of Circles
1	26703
2	26752
3	26626
4	26732
8	26896

Number of Threads	Number of Circles
16	26713
32	26766
40	26784
48	26721
56	26789
64	26847
72	26473
80	26730
88	26839
96	26590



The graph shows the number of circles for the run. Due to the nature of the problem (as described previously), there are differences as the number of threads increases.

r / L ratios

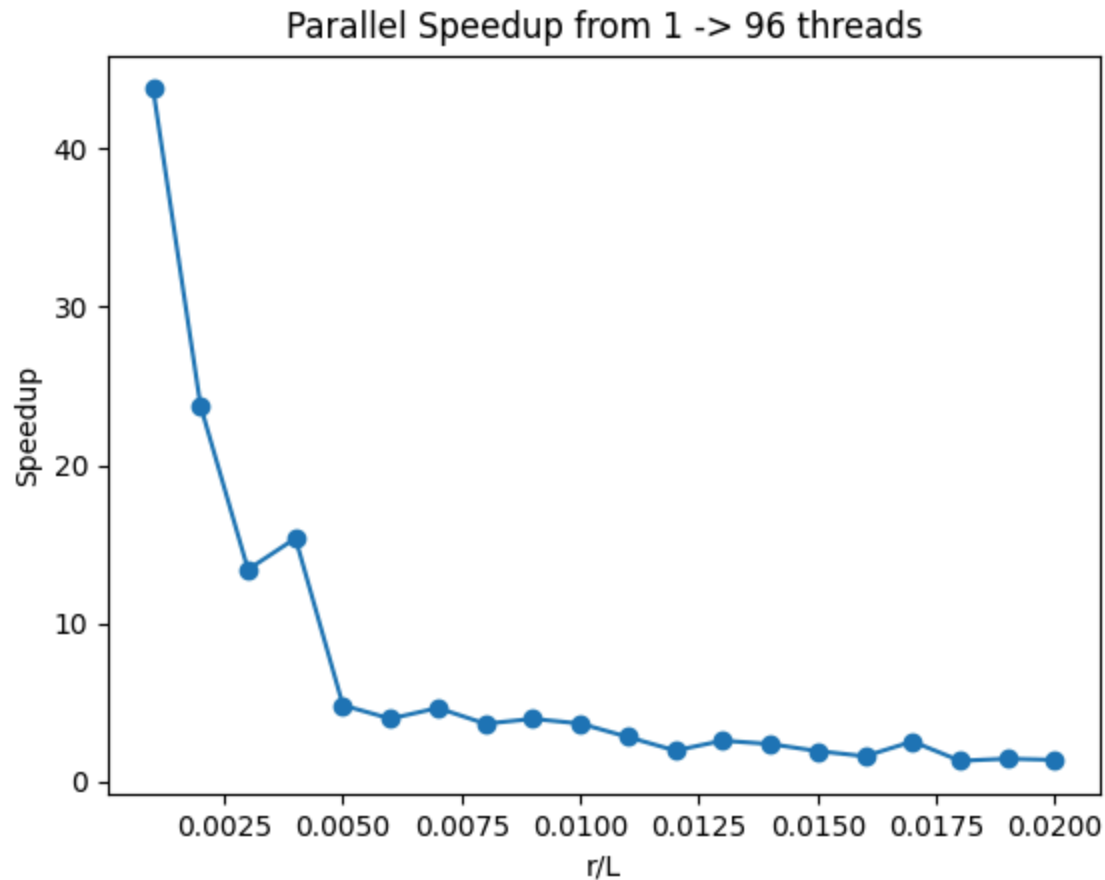
I ran the thread scaling test on `GViking` with user parameters:

```
r=VARIABLE
L=1000
sampling_frequency = 100000 = 1e5
seed=18293
```

using a large amount of job scripts and run scripts (see `previous_job_scripts`, `previous_run_scripts`). These scripts control the changing of the number of threads, creating the directories for results and using `make` to `make parallel` or `make parallel_vik` (depending on use). All relevant flags can be found in the

Makefile. I decided to iterate through r from 1 to 20 while keeping L=1000.

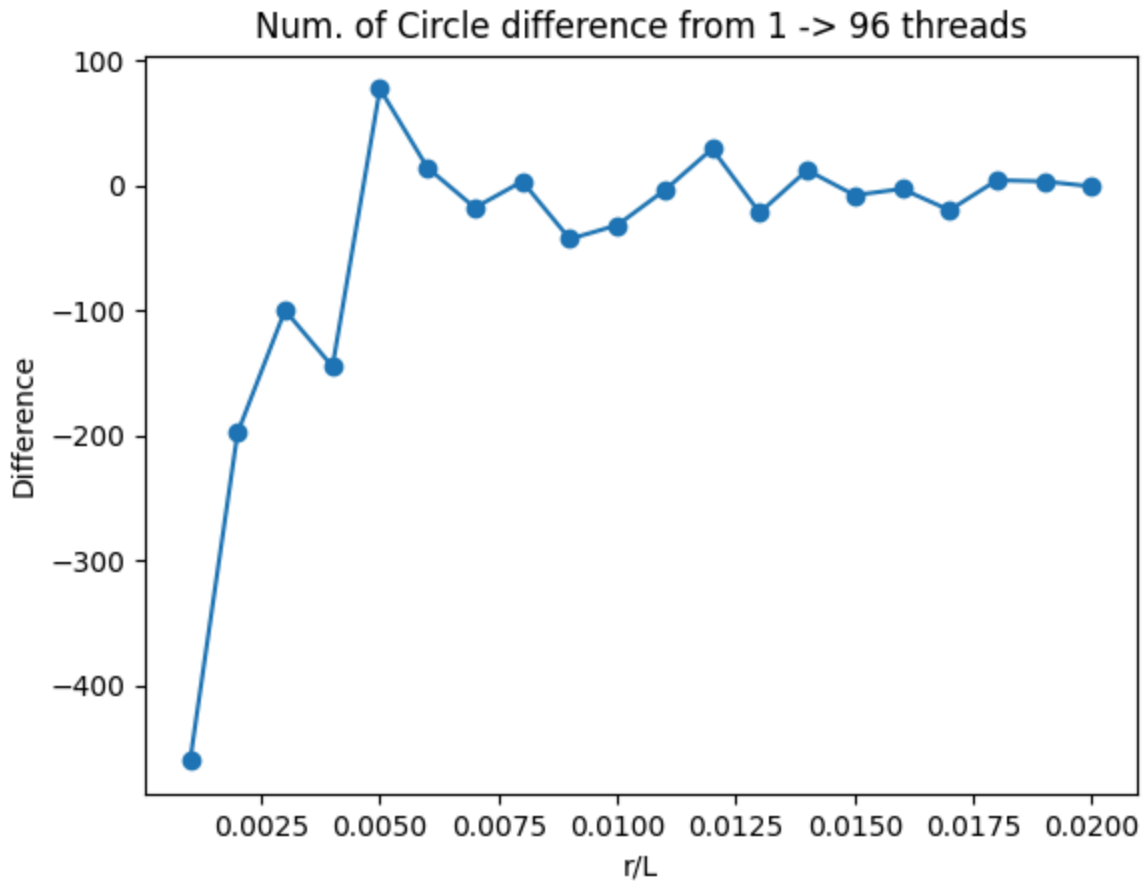
r	Time (1 thread) (s)	Time (96 threads) (s)	Speedup (1 -> 96 threads)
1	615.297813	14.054726	43.78
2	103.979342	4.377201	23.75
3	25.407766	1.893825	13.42
4	16.755429	1.088905	15.39
5	4.915592	1.015496	4.84
6	3.813935	0.954506	4.00
7	3.2491	0.695557	4.67
8	2.208212	0.600623	3.68
9	1.859043	0.466592	3.98
10	1.296213	0.351444	3.69
11	1.271669	0.44734	2.84
12	0.55206	0.278407	1.98
13	0.967813	0.37125	2.61
14	0.840325	0.350636	2.40
15	0.56483	0.289351	1.95
16	0.621947	0.382765	1.62
17	0.481745	0.188487	2.56
18	0.387136	0.289924	1.34
19	0.339204	0.231364	1.47
20	0.321071	0.23071	1.39



The graph above shows the parallel speedup for the different ratios. As expected, the original parameters $L=500$, $r=1.234$ was not a big enough problem for the parallel code to get the best speedup. At the smallest ratio ($L=1000$, $r=1$), there is a $\sim 43x$ speedup. This indicates that these parameters could fully utilise the parallel threading, and the problem size was large enough, noting Gustafson's law. However as the ratio increases, the effective speedup becomes quite small, which makes sense as the problem size is a lot smaller.

r	Num of Circles(1 thread)	Num of Circles (96 threads)	Difference (96 threads - 1 thread)
1	166986	166526	-460.00
2	41159	40961	-198.00
3	17869	17769	-100.00
4	9971	9826	-145.00

r	Num of Circles(1 thread)	Num of Circles (96 threads)	Difference (96 threads - 1 thread)
5	6223	6300	77.00
6	4301	4315	14.00
7	3164	3146	-18.00
8	2393	2396	3.00
9	1886	1843	-43.00
10	1506	1474	-32.00
11	1235	1231	-4.00
12	980	1009	29.00
13	893	871	-22.00
14	747	759	12.00
15	639	631	-8.00
16	571	568	-3.00
17	494	474	-20.00
18	438	442	4.00
19	384	387	3.00
20	343	342	-1.00



The graph above shows the difference in the number of circles at different ratios. The differences decrease as the ratio increases, which is explained by the difference in available space for the ratios. As there are more rng's with 96 threads, and a large amount of space in general to fill due to the small ratio, there is a higher chance that the new rng's either fail to fill a void or can fill a void.